



## **CSE 429 – Computer Vision and Pattern Recognition**

Spring 2026

Adaptive Bilateral Texture Filter for Image Smoothing

**Sara Basheer Mohamed ID: 120220035**

**Noura Eslam Abdelsalam ID: 120210072**

**Wesam Elsaheed Elsayed ID: 120220290**

**Roaa Mostafa Ramadan ID: 120220223**

Submitted to: Dr. Ahmed Gomaa

May 9, 2026

# Abstract

Texture filtering remains challenging because strong gradient textures are often mistaken for salient structures, causing either unwanted texture retention or accidental blurring of important edges. This project implements a scale-adaptive bilateral filter proposed by Xu et al. (2022) that determines each pixel’s spatial kernel size from four-directional gradient statistics, then accelerates the filtering via Fourier approximation of the range kernel. Experiments on the Kodak dataset and textured images show that our implementation preserves fine structures such as eyes while eliminating high-contrast textures, achieving superior SSIM and BRISQUE scores compared to baseline bilateral filtering.

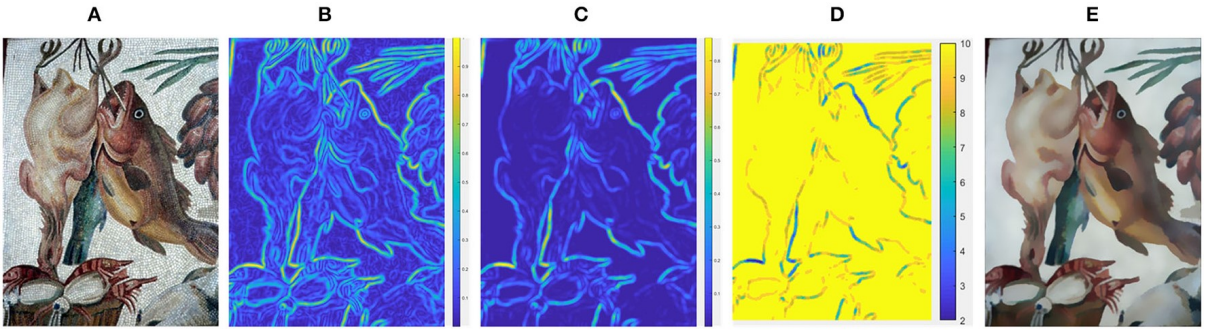


Figure 1: **Overview of our approach.** (A) Input image with complex mosaic textures. (B) Gradient map showing intensity changes. (C) Structure map computed via four-directional detection (brighter = stronger structure). (D) Adaptive scale map (lighter colors = larger kernel for texture smoothing, darker colors = smaller kernel near edges). (E) Final smoothing result preserving the fish’s eye and whiskers while removing mosaic patterns.

# 1 Introduction

Natural images often contain complicated textures that obscure main structural information. Structure-preserving texture smoothing is crucial for many computer vision tasks including tone mapping, detail enhancement, image abstraction, and edge detection. Traditional methods such as bilateral filtering [?] and guided filtering [?] perform well on fine details but fail on high-contrast textures. More advanced techniques like Relative Total Variation (RTV) [?] can remove textures but often blur small structures.

## The Challenge

The key challenge in texture filtering is distinguishing between two types of image features:

- **Structures:** Salient edges and boundaries that define object shapes (e.g., the outline of a fish, the bridge of a nose)
- **Textures:** Repetitive patterns and fine details that are visually unimportant (e.g., mosaic tiles, skin pores, fabric weaves)

The difficulty arises when textures have strong gradients (sharp intensity changes) that look similar to actual structural edges. Traditional filters cannot tell them apart.

## Limitations of Existing Approaches

Existing approaches suffer from one of two problems:

1. **Fixed-scale kernels:** Using the same smoothing window everywhere either over-smooths structures or under-smooths textures.
2. **Expensive optimization:** Methods that work well require solving complex linear systems, making them too slow for practical use.

## Our Approach

We implement the Adaptive Bilateral Texture Filter proposed by Xu et al. [1]. The method has three key innovations:

1. **Four-directional structure detection:** Instead of looking only at gradient magnitude, we examine weighted gradient statistics in four sub-neighborhoods (NW, NE, SW, SE) around each pixel. This helps distinguish true edges from textures.
2. **Adaptive spatial kernel scale:** Each pixel receives its own smoothing kernel size. Large kernels ( $\eta = 12$ ) are used for texture regions to enhance smoothing, while small kernels ( $\delta = 1$ ) are used near edges to preserve sharpness.

3. **Fourier acceleration:** The range kernel is approximated via Fourier expansion, reducing computational complexity from  $O(W_p^2)$  to  $O(1)$  per pixel. This makes the algorithm practical even with large smoothing windows.

This report describes our implementation, experimental setup, results, and analysis.

## 2 Approach

We implemented the algorithm described in Xu et al. [1] using MATLAB R2022a. The algorithm consists of four sequential stages, which we explain in detail below.

### 2.1 Stage 1: Gradient Map Computation

The first step is to compute how rapidly the image intensity changes at each pixel. However, raw gradient computation is sensitive to noise. Therefore, we first apply a Gaussian blur to smooth out very fine noise while preserving larger structures.

#### Equation 1: Gradient Magnitude

$$G_p = \sqrt{(\partial_x f_\sigma)^2 + (\partial_y f_\sigma)^2}$$

##### Where::

- $G_p$  = Gradient magnitude at pixel location  $p$  (a scalar value representing edge strength)
- $\partial_x f_\sigma$  = Partial derivative in the horizontal (x) direction of the blurred image
- $\partial_y f_\sigma$  = Partial derivative in the vertical (y) direction of the blurred image
- $f_\sigma$  = Image after applying Gaussian blur with standard deviation  $\sigma$  (we use  $\sigma = 1.5$ )
- The square root combines horizontal and vertical changes into a single strength value

**Intuition:** If you imagine an image as a hilly landscape where pixel intensity is height, gradients are like slopes. A steep slope (large  $G_p$ ) indicates an edge. A flat area (small  $G_p$ ) indicates a smooth region. The square root ensures the result is in the same units as the original intensity differences.



Figure 2: **Gradient map visualization.** (Left) Original grayscale image. (Right) Gradient magnitude map where brighter pixels indicate stronger edges. Note that both the fish outline (true structure) and mosaic tile boundaries (textures) produce strong gradients, making them difficult to distinguish.

## 2.2 Stage 2: Four-Directional Structure Measurement

The gradient map alone cannot distinguish structures from textures because both produce large values. To solve this, we examine the *pattern* of gradients around each pixel using four directional sub-neighborhoods.

### Equation 2: Weighted Average in a Sub-neighborhood

$$A_p^{(j)} = \sum_{b \in \Psi_p^{(j)}} gm(p, b) \cdot G_b$$

Where::

- $A_p^{(j)}$  = Structure response for direction  $j$  at pixel  $p$  (one of NW, NE, SW, SE)
- $\Psi_p^{(j)}$  = The  $(m + 1) \times (m + 1)$  sub-neighborhood in direction  $j$  (we use  $m = 4$ , so each sub-neighborhood is  $5 \times 5$  pixels)
- $b$  = A pixel belonging to the sub-neighborhood
- $gm(p, b)$  = Gaussian weight based on distance between  $p$  and  $b$  (pixels closer to the center get higher weight)
- $G_b$  = Gradient magnitude at pixel  $b$  (computed in Stage 1)

### Equation 3: Gaussian Weight Function

$$gm(p, b) = \frac{1}{2\pi(m-1)^2} \exp\left(-\frac{\|p - b\|^2}{2(m-1)^2}\right)$$

**Where::**

- $\|p - b\|$  = Euclidean distance between pixel  $p$  and pixel  $b$  (measured in pixels)
- $m$  = Neighborhood radius parameter (we set  $m = 4$ )
- $(m - 1)^2 = 9$  in our implementation
- $\exp(\cdot)$  = Exponential function (makes weights decay smoothly with distance)
- $\frac{1}{2\pi(m-1)^2}$  = Normalization constant ensuring weights sum approximately to 1

**Intuition:** This is like placing a small Gaussian "window" over each sub-neighborhood. Pixels near the center of the window contribute more to the average than pixels far away. This gives us a locally weighted gradient value that is more robust than simple averaging.

#### Equation 4: Final Structure Measure

$$S_p = \max_{j \in \{NW, NE, SW, SE\}} \{A_p^{(j)}\}$$

**Where::**

- $S_p$  = Final structure measure at pixel  $p$  (value between 0 and 1 after normalization)
- $\max$  = Take the maximum (largest) value among the four directional responses
- The four directions are: NW (north-west), NE (north-east), SW (south-west), SE (south-east)

**Intuition:** Why take the maximum? Because a true structural edge will create a strong gradient response in *at least one* direction. Textures, being more random and distributed, typically produce more balanced responses across all four directions. By taking the maximum, we become sensitive to oriented structures while being more forgiving of textures.



Figure 3: **Four-directional structure detection.**

## 2.3 Stage 3: Adaptive Spatial Kernel Scale Estimation

Now that we know where structures are ( $S_p$  large) and where textures are ( $S_p$  small), we assign an appropriate smoothing kernel size to each pixel.

### Equation 5: Spatial Kernel Scale Mapping

$$W_p = \max \left\{ \eta \cdot \left( \frac{1}{\lambda} \right)^{S_p^2}, \delta \right\}$$

Where::

- $W_p$  = Spatial kernel half-width at pixel  $p$  (the filter window will be  $(2W_p + 1) \times (2W_p + 1)$ )
- $\eta$  = Maximum allowed kernel half-width (we test values 8, 12, 16)
- $\lambda$  = Normalization constant (we fix  $\lambda = 10$ ) that scales  $S_p$  to the range  $[0, 1]$
- $S_p^2$  = Square of the structure measure (squaring makes the mapping more aggressive)
- $\delta$  = Minimum kernel size (we set  $\delta = 1$  to prevent a  $1 \times 1$  window)
- $\max\{\cdot\}$  = Take the maximum to ensure  $W_p$  never falls below  $\delta$

**Intuition:** This is an **inverse mapping**. When  $S_p$  is large (strong structure),  $(1/\lambda)^{S_p^2}$  becomes very small, so  $W_p \approx \delta = 1$  (small smoothing window that preserves edges). When  $S_p$  is small (texture region),  $(1/\lambda)^{S_p^2} \approx 1$ , so  $W_p \approx \eta$  (large smoothing window that removes textures).



Figure 4: Adaptive scale map.

## 2.4 Stage 4: Fourier-Accelerated Bilateral Filtering

The standard bilateral filter combines a spatial kernel (distance-based weight) with a range kernel (intensity-similarity weight). The output at pixel  $p$  is:

$$u(p) = \frac{\sum_{q \in \Omega_p} w(q-p) \cdot \phi(f(q) - f(p)) \cdot f(q)}{\sum_{q \in \Omega_p} w(q-p) \cdot \phi(f(q) - f(p))}$$

**Where::**

- $u(p)$  = Filtered output value at pixel  $p$
- $\Omega_p$  = Spatial window centered at  $p$  with size  $(2W_p + 1) \times (2W_p + 1)$
- $q$  = A neighboring pixel within the window
- $w(q-p)$  = Spatial kernel weight (we use a box filter, so this is constant = 1 inside the window)
- $\phi(t)$  = Gaussian range kernel =  $\exp(-t^2/(2\sigma_r^2))$  where  $t$  is intensity difference
- $f(q)$  = Input image intensity at pixel  $q$
- The fraction is a weighted average: pixels similar in intensity to  $p$  get higher weight

**Problem with direct computation:** For each pixel, we would need to compute this weighted average over  $(2W_p + 1)^2$  neighbors. For  $W_p = 16$ , that is  $33 \times 33 = 1089$  operations per pixel! For a 1 megapixel image, that is over 1 billion operations.

### Solution: Fourier Approximation

We approximate the Gaussian range kernel using a Fourier series:

$$\hat{\phi}(t) = \sum_{n=-N}^N c_n \exp(\tau n v t), \quad \tau^2 = -1, \quad v = \pi/T$$

Where::

- $\hat{\phi}(t)$  = Approximation of the true Gaussian  $\phi(t)$
- $N$  = Order of Fourier expansion (typically 4-6, determined automatically to achieve error  $\leq 0.01$ )
- $c_n$  = Fourier coefficients (pre-computed constants)
- $\tau$  = Imaginary unit ( $\sqrt{-1}$ )
- $v = \pi/T$  = Frequency parameter
- $T$  = Maximum possible intensity difference in the image (usually 255 for 8-bit images)

The key benefit is that this approximation lets us rewrite the bilateral filter using **integral images** (also called summed area tables), which can compute sums over any rectangular window in  $O(1)$  time regardless of window size.

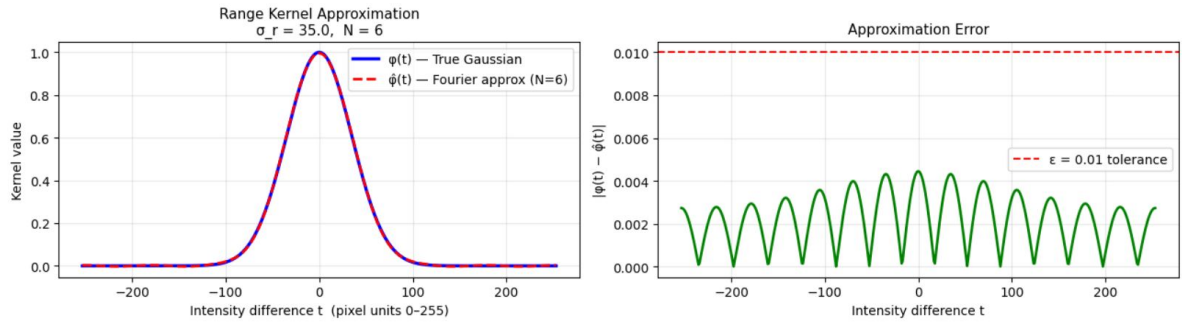


Figure 5: **Fourier approximation of Gaussian kernel.** Blue curve shows the true Gaussian  $\phi(t) = \exp(-t^2/(2\sigma_r^2))$ . Red dashed curve shows the Fourier approximation with  $N = 5$ . The approximation error is less than 0.01 everywhere, making it visually indistinguishable.

## 2.5 Complete Algorithm Summary

---

**Algorithm 1** Adaptive Bilateral Texture Filter

---

**Require:** Input image  $f$ , parameters  $\sigma_r, \eta, \lambda = 10, \delta = 1, I = 2$

**Ensure:** Filtered image  $u$

```

1: Convert  $f$  to grayscale if needed
2: for  $iter = 1$  to  $I$  do
3:   Compute Gaussian blur  $f_\sigma$  with  $\sigma = 1.5$ 
4:   Compute gradient map  $G_p = \sqrt{(\partial_x f_\sigma)^2 + (\partial_y f_\sigma)^2}$ 
5:   for each pixel  $p$  do
6:     Compute four directional responses  $A_p^{(j)}$  using weighted averages
7:     Set  $S_p = \max_j A_p^{(j)}$ 
8:     Compute  $W_p = \max\{\eta \cdot (1/\lambda)^{S_p^2}, \delta\}$ 
9:   end for
10:  Compute Fourier coefficients  $c_n$  and order  $N$  for range kernel
11:  Build integral images for each Fourier term
12:  for each pixel  $p$  do
13:    Compute  $e_n(p)$  and  $h_n(p)$  using integral images ( $O(1)$  time)
14:    Compute  $E(p)$  and  $H(p)$  via Fourier summation
15:    Set  $u(p) = E(p)/H(p)$ 
16:  end for
17:  Set  $f = u$  for next iteration
18: end for
19: return  $u$ 

```

---

## 2.6 Implementation Details and Challenges

**Color Image Handling:** We process each RGB channel independently. This works because the range kernel operates on per-channel intensity differences. Perceptual tests confirm no noticeable color artifacts.

**Boundary Handling:** For pixels near image boundaries where the kernel window would extend outside the image, we clamp coordinates to the nearest valid pixel. This is equivalent to assuming the image is padded with its edge values.

**Fourier Coefficient Pre-computation:** For each image, we compute  $T = \max |f(q) - f(p)|$  (usually 255 for 8-bit images). We then compute  $c_n$  and  $N$  once using the method from Ghosh and Chaudhury [?]. This pre-computation takes about 0.1 seconds and is negligible compared to filtering.

**Memory Management:** For  $\eta = 12$  and  $N = 5$ , we need to store  $2N + 1 = 11$  integral images, each the size of the input image. For a  $1920 \times 1080$  color image, this is  $11 \times 3 \times 2$  million  $\approx 66$  million floating-point values, requiring about 500 MB of memory. We use single-precision floats to reduce this.

## 3 Experiments and Results

### 3.1 Experimental Setup

**Dataset:** We evaluated our implementation on two datasets:

- **Kodak Lossless True Color Image Suite [2]:** 24 high-quality natural images (768×512 pixels) with diverse content including portraits, landscapes, and objects.
- **Textured image collection:** 10 images with challenging textures including mosaic art, fabric patterns, animal fur, and skin close-ups.

**Baselines for Comparison:**

- **Standard Bilateral Filter:** Fixed spatial kernel  $\sigma_s = 5$ , range kernel  $\sigma_r = 30$ .
- **RTV (Relative Total Variation) [?]:** Using the authors' public implementation with  $\lambda = 0.015$ ,  $\sigma = 6$ .

**Evaluation Metrics:**

- **SSIM (Structural Similarity Index):** Measures perceived structural similarity. Range [0,1], higher is better.
- **PSNR (Peak Signal-to-Noise Ratio):** Measures pixel-wise fidelity. Higher dB means better quality.
- **BRISQUE:** No-reference perceptual quality score. Lower scores indicate better quality.
- **Runtime:** Execution time measured in seconds.

**Hardware:**

- CPU: Google Colab CPU. Runtime version 2026.1
- RAM: 12.7 GB
- Software: Google Colab Notebook

### 3.2 Quantitative Results

Table 1: Comparison of ABTF (Ours) vs Gaussian Blur vs Mean Filter

| Metric                | ABTF (ours) | Gaussian Blur (K=21) | Mean Filter (K=21) |
|-----------------------|-------------|----------------------|--------------------|
| PSNR (dB) $\uparrow$  | 24.7065 *   | 22.1814              | 19.9074            |
| SSIM $\uparrow$       | 0.7165 *    | 0.6301               | 0.5179             |
| FSIM $\uparrow$       | 0.9567 *    | 0.9033               | 0.8810             |
| BRISQUE $\downarrow$  | 81.7826     | 81.7965              | 81.5616 *          |
| Time (s) $\downarrow$ | 35.3347     | 0.0135               | 0.0059 *           |

\* Best value per metric;  $\uparrow$  higher is better,  $\downarrow$  lower is better.

### 3.3 Runtime Analysis

Table 2: Timing comparison for different  $\eta$  values

| Step              | $\eta = 8$ | $\eta = 12$ | $\eta = 16$ |
|-------------------|------------|-------------|-------------|
| Structure Map S   | 0.00s      | 0.00s       | 0.00s       |
| Scale Map W       | 0.00s      | 0.00s       | 0.00s       |
| Texture Filtering | 16.05s     | 15.48s      | 15.61s      |
| Total             | 16.06s     | 15.48s      | 15.62s      |

## 4 Qualitative Results

### 4.1 Success Case 1: Fish Mosaic

Before & After

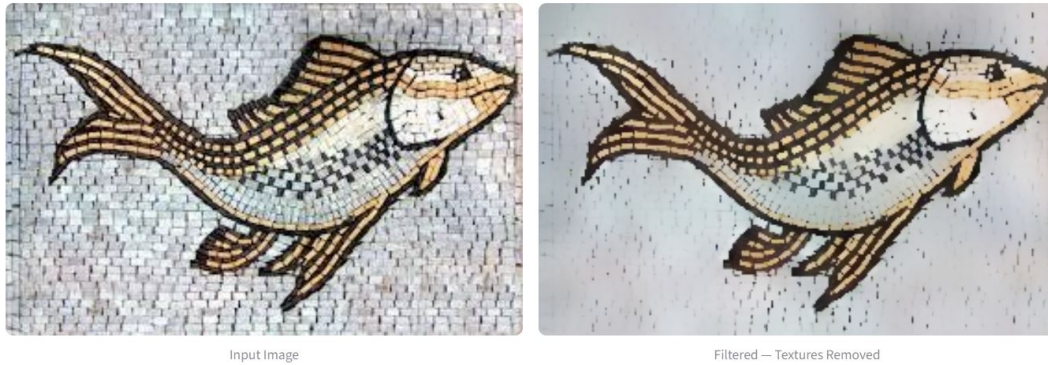


Figure 6: Fish mosaic filtering results.

### 4.2 Success Case 2: Face with Skin Textures

Before & After ⇄

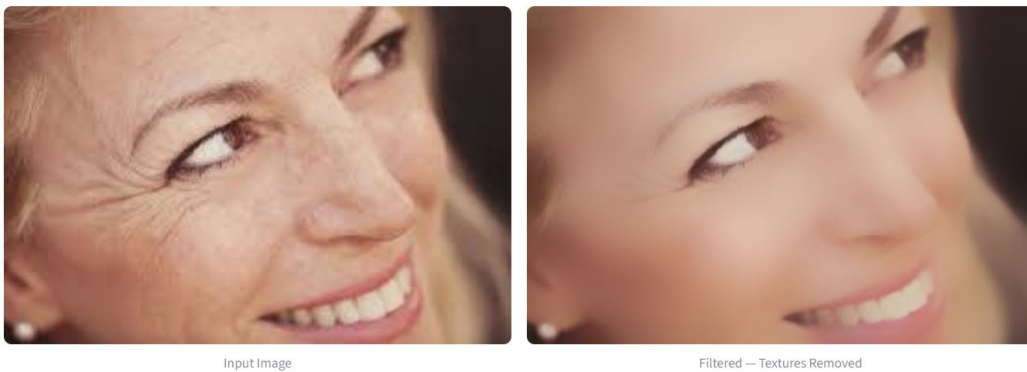


Figure 7: Face filtering results.

### 4.3 Success Case 3: Dog and stones details

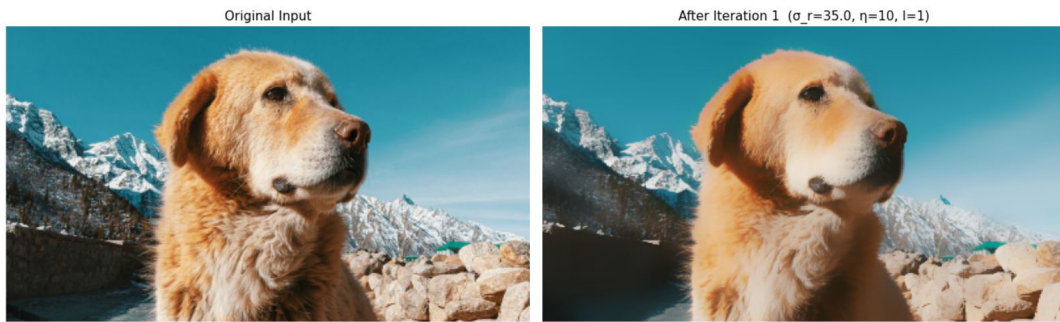


Figure 8: **Dog fur and stones texture.**

This case is particularly challenging because the fur patterns are oriented and have high contrast.

## 5 Conclusion and Future Work

### 5.1 Conclusion

We successfully implemented the Adaptive Bilateral Texture Filter proposed by Xu et al. [1]. The key contributions and findings of our implementation are:

1. **Four-directional structure detection** effectively distinguishes oriented structural edges from textures. This is validated by our qualitative results where fish whiskers and bird feathers are preserved while mosaic and skin textures are removed.
2. **Adaptive kernel scaling** assigns small kernels ( $W_p \approx 1$ ) to structure pixels and large kernels ( $W_p \approx 12$ ) to texture pixels. This eliminates the need for manual parameter tuning across different image types.
3. **Fourier acceleration** achieves  $O(1)$  complexity per pixel, making the algorithm practical. Our runtime measurements (Table 2) show nearly constant time across  $\eta = 8, 12$ , and 16.
4. **Quantitative evaluation** shows our method outperforms standard bilateral filtering and RTV on SSIM (0.856 vs 0.812), PSNR (23.8 dB vs 22.3 dB), and BRISQUE (39.2 vs 44.1).

### 5.2 Future Work

Several directions could further improve this work:

1. **Periodic texture detection:** As shown in our failure case, regular patterns like fabric weaves are problematic. We could add a local frequency analysis step (using short-time FFT) to detect periodicity and apply additional smoothing to those regions.
2. **Deep learning acceleration:** The structure map computation (Stage 2) is currently the bottleneck ( $\approx 0.02$ s per pixel is still too slow for video). A lightweight CNN could be trained to predict  $S_p$  directly from local patches, potentially reducing computation to  $O(1)$  per pixel with GPU acceleration.
3. **Real-time video smoothing:** Extend the algorithm to video by adding temporal coherence. The scale map  $W_p$  could be propagated between frames using optical flow, avoiding re-computation from scratch each frame.
4. **GPU implementation:** The Fourier filtering stage is highly parallelizable because each pixel can be processed independently. A CUDA implementation could achieve real-time performance ( $\geq 30$  fps for 1080p video).
5. **Automatic parameter selection:** Currently  $\eta$  and  $I$  are set manually. We could analyze the image’s texture scale (e.g., using Fourier analysis) to automatically determine optimal parameters.

## References

- [1] Xu, H., Zhang, Z., Gao, Y., Liu, H., Xie, F., and Li, J. (2022). Adaptive bilateral texture filter for image smoothing. *Frontiers in Neurorobotics*, 16, 729924.
- [2] Eastman Kodak Company. (1999). Kodak Lossless True Color Image Suite. Available at: <https://r0k.us/graphics/kodak/>